OpenCode Memory — Complete Project and Feature Guide

Autor: OpenCode Memory Project Documentation | Data: 28/09/2026

1. Purpose and Scope

OpenCode Memory is a local observational memory system for OpenCode sessions. It is not a remote knowledge service and it does not guarantee that model-generated observations are true. It retains a redacted snapshot of completed conversation, asks an observer model to extract attributable observations, injects relevant memory into later turns, and gives the user a reviewable dashboard.

The system is designed around four principles:

- Local-first storage in SQLite, with an optional Markdown vault and scrub-aware backups.
- Attribution: every observation should be traceable to retained source, or explicitly marked as a user write.
- Split trust: the agent may propose; humans approve shared, high-impact, and mastery claims.
- Fail closed: invalid observer output is dropped, retried, or quarantined; chat is never blocked if injection misses its deadline.

Current product boundaries: learning and research state is workspace-wide rather than project-specific; embeddings are optional and off by default; the dashboard exists only while a plugin process owns the loopback port; the desktop Memory tab is a thin, separately built OpenCode fork integration.

2. System Architecture

2.1 Packages

The repository is a Bun workspace with four packages plus an optional sibling OpenCode desktop fork.

- packages/core — runtime-neutral TypeScript. Owns policy, SQLite, redaction, ingestion, chunking, observer/reflector validation, retrieval, injection, review, learning, research, artifacts, vault, forget, and backup/restore. Uses bun:sqlite under Bun and node:sqlite otherwise.
- packages/plugin — OpenCode plugin. Captures session/message lifecycle, injects memory into the system prompt, registers hidden observer/reflector agents, exposes tools and slash commands, and serves the loopback dashboard/API.
- packages/dashboard — Solid UI bundled into the plugin. Serves /app (full dashboard) and /embed (Memory tab iframe).
- packages/evals — hermetic acceptance tests and an opt-in real-provider evaluation.
- Sibling OpenCode fork — Memory side-panel tab that embeds /embed over a constrained postMessage bridge.

2.2 Runtime flow

One plugin process binds 127.0.0.1:4777. Other OpenCode windows share the same SQLite database and route long-running commands through the owning instance using heartbeats, leases, jobs, and a command queue. Hidden om-observer and om-reflector child sessions extract structured JSON; they are denied every tool except StructuredOutput.

SQLite uses WAL, an 8-second busy timeout, foreign keys, secure_delete, synchronous=NORMAL, and FTS5 when available. If FTS5 is missing, search falls back to in-process BM25.

2.3 Data model groups

- Identity: projects, sessions, fork_map, instances.
- Retained source: source_messages, source_parts, ingest_state.
- Observation pipeline: chunks, coverage, observations, observation_sources, observation_events, session_state.
- Review and lifecycle: review_items, suppressions, suppression_shingles, context_snapshots.
- Coordination: jobs, commands, operations, runtime_state.
- Reflection/vault: reflections, topics, topic_generations, vault_outbox, journey_entries.
- Retrieval: observations_fts (when FTS5 exists) and embeddings.
- Learning: learning_concepts, learning_attempts, review_cards.
- Research: research_sources, research_claims, claim_evidence.
- Artifacts: artifacts.

3. Capture and Observation Pipeline

3.1 Capture

The plugin snapshots completed OpenCode messages into a redacted local source store.

- User messages are immediately complete.
- Assistant messages are captured when completed, or after the stale-incomplete timeout (default 30 minutes).
- Reasoning is omitted by default and replaced by an omission marker.
- Memory-tool output is not recursively retained, to avoid recall loops.
- Ordinary tool output is bounded (default 64,000 characters, middle-truncated).
- Internal om-* worker sessions are never observed. Ordinary subagents are excluded unless observeSubagents is enabled.

Redaction runs before source storage. Rules cover private keys, cloud/API tokens, JWT/bearer values, URL credentials, secret-like assignments, and high-entropy candidates. Redaction is heuristic: novel secrets may pass, and ordinary high-entropy text may be over-redacted.

3.2 Chunking and activation

Captured text is cut into fixed durable chunks at the buffer threshold (default 6,000 tokens). Automatic model observation waits until a session's total unobserved work reaches the activation threshold (default 30,000 tokens). Idle flush (90s), max unobserved age (10 minutes), startup recovery, and explicit Flush always process smaller sessions so they are not stranded.

3.3 Observer

The observer model is independent of the chat model. It is configured under runner.model (providerID/modelID). If omitted, OpenCode small_model is used, then the current session model. Default runner.kind is opencode-session: a hidden child session titled [memory] observer with structured JSON-schema output.

The observer must produce observations with kind, content, confidence, evidence type, priority, source message refs, optional verbatim quote, and a scope hint. Core validation checks:

- source refs exist in the chunk;
- quotations are verbatim;
- user-sourced kinds (preference, learning-preference, goal, personal-fact) require user attribution;
- instruction-like text is refused without an attributable user source;
- forgotten-content suppressions cannot be recreated;
- domain payloads match the kind.

Invalid output is retried; repeated transient failure quarantines the chunk. Authentication failure pauses workers until credentials are fixed and the user resumes. Daily cost/token caps reserve budget before each run.

3.4 Reflection and vault

When a project or workspace crosses the reflection threshold (default 40,000 new durable tokens or 25 new durable observations), the reflector consolidates observations into topic notes. It may create or merge topic generations, condense observations, supersede stale facts, append journey entries, or propose promotion to a broader scope. Topic bullets cite observation IDs. Mastery assertions are not allowed from the reflector.

The Markdown vault is an asynchronous projection, not the source of truth:

- workspace INDEX.md and JOURNEY.md;
- topic Markdown under workspace/project folders;
- front matter with topic ID, generation, and body hash.

External file edits open a conflict instead of being overwritten. Vault paths are checked lexically and after symlink/junction resolution; UNC and device paths are refused.

4. Scopes, Kinds, and Trust

4.1 Scopes

The workspace is the sharing boundary. Domain (general, learning, research) is classification, not access control.

- session — readable within the owning project. Unreviewed direct agent writes are readable only in their own session.
- project — readable only in that project.
- workspace — readable by projects assigned to the same workspace.
- user — readable in every workspace.

Pending-review observations are readable only from their originating session. Source recall is narrower than memory visibility: a user-scoped fact can be readable across workspaces while its source remains hidden.

4.2 Observation kinds

General: fact, preference, goal, decision, task-state, event, personal-fact, project-context, tool-outcome.

Learning: prior-knowledge, quiz-attempt, demonstrated-skill, misconception, hint-required, teaching-intervention, retention-check, learning-preference, next-review.

Research: idea, hypothesis, claim, source, quotation, evidence, counter-evidence, experiment, open-question, conclusion, superseded-conclusion.

Plus artifact and condensed.

High-impact kinds that always need human review: personal-fact, conclusion, superseded-conclusion. Explicit mastery claims also always go to review.

4.3 Agent versus user writes

Agent writes (memory_remember and domain tools):

- may become active only at session/project scope;
- become pending review at workspace/user scope;
- also go to review for user-sourced or high-impact kinds and mastery claims;
- cannot replace workspace/user memories;
- cannot write instruction-like text without a user source.

Dashboard/user writes can enter any scope directly. They are still redacted, validated, and refused if the target session was forgotten, excluded, or deleted.

5. OpenCode Slash Commands

These commands are templates that instruct the primary agent to call the corresponding tools.

5.1 /remember

Asks the agent to call memory_remember with the provided text, then confirm what was stored. The user may include kind and scope in natural language; the agent maps them onto tool arguments.

5.2 /recall

Asks the agent to search with memory_search, then call memory_recall_source on the most relevant IDs so answers quote the exact retained, redacted snapshot rather than paraphrasing.

5.3 /memory-status

Calls memory_status and summarizes capture, observer, SQLite/FTS, pause, and review state. Used to diagnose buffering, activation, last errors, and whether observation has ever succeeded.

5.4 /review-due

Calls learning_due_reviews and offers to quiz the user on concepts whose FSRS cards are due.

5.5 /claims

Calls research_claims and, if there are contested claims, research_conflicts, focusing on the user's argument.

6. Agent Tools in Detail

All of the following tools are registered by the plugin for the primary agent. Hidden om-* workers cannot use them.

6.1 memory_status

Shows observational memory status for the current session. Reports:

- derived state: unknown, paused, observing, reflecting, buffering, or idle;
- pause reason (manual or auth);
- unobserved tokens versus activation threshold, buffer tokens, captured tokens;
- observation count and observation tokens;
- current task and suggested next action from the last observer run;
- last error, last success, and last failure timestamps;
- FTS5 availability, SQLite driver, and open-review count.

No arguments. Safe to call at any time.

6.2 memory_observations

Lists recent observations visible in the current session/project access context.

Arguments:

- limit (optional, 1–50, default 20)
- kinds (optional comma-separated kind filter)

Each line includes content, observation ID, kind, scope, status, and confidence. Used by the agent to pick IDs for recall, supersede, or forget.

6.3 memory_search

Lexical or hybrid search over observations in scope.

Arguments:

- query (required)
- limit (optional, 1–30, default 10)

SQLite FTS5 provides BM25 when available. Optional embeddings blend vector rank with text rank. Results include score. Search can include superseded history from the dashboard; the agent tool returns currently listable rows.

6.4 memory_recall_source

Recalls the exact retained, redacted snapshot behind an observation or a session sequence range. This is not a live fetch of OpenCode's current transcript.

Arguments:

- observationId (optional)
- sessionId, fromSeq, toSeq (optional range)
- context (optional, 0–10 extra messages around the range)

The result includes a guarantee string, cited ranges, per-message status, and per-part markers: forgotten, redacted counts, omitted characters, truncated. Pages are bounded to 40 messages and 80,000 characters. A truncated page ends with the exact nextCursor arguments to pass back unchanged.

Recall may contain [REDACTED:…] markers, omission/truncation markers, [forgotten] parts, and reverted/removed status. A directly written memory may have no source.

6.5 memory_remember

Writes a memory observation as actor agent.

Arguments:

- content (required)
- kind (optional)

• scope: session, project, workspace, or user (optional)
• payload (optional structured fields for the kind)

Payload fields (unused fields are dropped): key, value, concept, question, answer, evaluation, hints, independent, mastery, reviewAt, claim, stance, url, doi, title, publishedAt, accessedAt, locator, quotation, conditions, path.

Returns the new ID, resolved scope, and status (active or pending_review). Shared/high-impact writes wait for human review.

6.6 memory_supersede

Replaces an existing observation with corrected content. The old row remains in lifecycle history; the new row becomes current.

Arguments:

• id (required)
• content (required)
• reason (optional)

The agent cannot supersede shared (workspace/user) memories or pending-review rows. Users can supersede from the dashboard.

6.7 memory_forget

Forgets observations by ID after an OpenCode permission prompt, or lists candidates without deleting when only a text query is given.

Arguments:

• ids (optional array)
• query (optional; lists candidates if ids is empty)
• reason (optional)

Agent forget is scope-bounded to the current project/workspace access context, keeps retained sources by default, reports skipped IDs, and leaves tombstones so observation cannot recreate the content. Default permission is ask unless the user configured a specific rule.

Dashboard forget is different: it is a confirmed user-global action and can also erase cited conversation parts when requested.

6.8 memory_flush

Flushes unobserved session text through the observer immediately, bypassing the activation threshold. Waits up to 120 seconds. If the user aborts, the observer keeps running in the background.

Returns chunk count, whether the wait completed, and how many observations were inserted. Used after /memory-status shows a large backlog below or above activation, or after changing the observer model.

6.9 learning_due and learning_due_reviews

Aliases for the same tool. Lists learning concepts due for FSRS review in the current workspace. Returns concept name, ID, and due timestamp, or "No reviews due."

6.10 learning_record_attempt

Records a learner quiz or retention attempt. The quoted answer must appear in the latest user message as a word-boundary match of at least three words. Recalled evidence does not create a new attempt. Hints force independent=false. Not treated as human-verified.

Arguments:

- concept (required)
- answer (required)
- question (optional)
- evaluation: correct, partial, incorrect (optional, default partial)
- hints (optional, 0–50)
- independent (optional boolean)
- kind: quiz-attempt, retention-check, or demonstrated-skill (optional, default quiz-attempt)

Agent-recorded attempts stay session-scoped or go to review; they do not update workspace mastery tables until a human approves or promotes them.

Mastery model after promotion:

- introduced — concept exists with no qualifying attempts
- practicing — attempts exist
- demonstrated — at least one independent correct attempt
- retained — at least two independent correct attempts on distinct days, spread by at least three days, with no open misconception
- needs-retention-check — demonstrated/retained evidence is older than 45 days

6.11 research_claims

Lists up to 20 research claims in the workspace with status and support/contradict counts, then lists conflicts if any exist.

Claim status is derived from active workspace/user evidence:

- open — no linked evidence
- supported — supporting evidence only
- refuted — contradicting evidence only
- contested — both
- superseded — explicitly superseded conclusion

This is an evidence organizer, not a fact checker.

6.12 research_conflicts

Lists only contested claims (the conflicts view), up to 20.

6.13 research_claim

Records an agent-observed research claim. Evidence is marked agent-observed, not human-verified.

Arguments: claim (required), plus optional url, doi, title, quotation, locator, conditions.

The row is written through remember at session scope (or review). It does not enter workspace claim tables until approved/promoted.

6.14 research_evidence

Records supporting or contradicting evidence for a claim. Marked agent-observed, not human-verified.

Arguments: claim (required), stance support|contradict (default support), quotation, url, doi, locator, content.

URLs and DOIs identify sources. Quotations and locators are retained when supplied and validated where applicable.

6.15 artifact_register

Indexes an SVG or Mermaid artifact already written to disk in the current project.

Arguments:

- path (required; .svg, .mmd, or .mermaid)
- title (optional)
- concepts (optional string array)

Path safety:

- must resolve inside the current project after symlink/junction resolution;
- UNC and device paths are refused;
- maximum file size 2 MB;
- stored render content is capped at 200,000 characters; larger files remain listed as content not retained.

The plugin also auto-indexes .svg/.mmd/.mermaid files touched by write, edit, multiedit, apply_patch, and patch tools, plus fenced Mermaid/SVG/XML and raw SVG in completed assistant text. The dashboard sanitizes SVG before display.

7. Prompt Injection, Compaction, and Pruning

7.1 Automatic injection

When inject.enabled is true, the plugin builds a bounded memory block for the current turn under inject.deadlineMs (default 250 ms). Budgets cover profile (user), workspace, project, session, task, and query recall. The block is wrapped as a stable opencode-memory hash element and reused for every step of that turn unless memory is invalidated (forget, edit, review, config change, pause).

Two consecutive deadline overruns cause a ten-minute per-session injection cooldown. Injection failure never blocks chat; the turn proceeds without memory.

7.2 Compaction note

On OpenCode compaction, the plugin can attach a short observational-memory note: current task, suggested next, recent INDEX/JOURNEY snippets, and recent session observations, truncated to an 800-token budget.

7.3 Optional pruning

When pruning.enabled is true, already-covered older messages can be removed from the model request (not from retained source). keepRecentTurns defaults to 4. This is disabled by default because it changes what the primary model sees.

8. Dashboard Features

The full UI is http://127.0.0.1:4777/app. HTML is served with no-store, a restrictive CSP, and an injected local token. /api/* requires X-OM-Token. Host must be 127.0.0.1 or localhost on the configured port.

8.1 Overview

Service state (idle, buffering, observing, reflecting, paused, error), connected OpenCode instances, open-review count, due learning reviews, recent runs, daily cost/token gauges, pause/resume.

8.2 Timeline

Choose a session; filter by scope, kind, evidence, or status; inspect sources, events, and supersession. Actions: remember, pin, edit, supersede, recall, forget. Source links can navigate back to the originating OpenCode message from the Memory tab.

Pinning improves rank and prevents freshness decay. Edit is an in-place user correction. Supersede keeps the old statement in lifecycle history.

8.3 Search

Lexical/hybrid search including superseded history, with paged exact-source recall via Load more / nextCursor.

8.4 Knowledge

Reflected topic notes, generations, approval, rollback, and Markdown vault conflicts. "Keep my file edit" imports/sanitizes external Markdown as an external edit (not human approval of the underlying observations). "Overwrite file from memory" replaces the file with the stored topic.

8.5 Learning

Workspace concept map, evidence-based mastery level, independent/assisted/incorrect counts, FSRS calendar, and misconceptions. The focused Memory-tab card appears when the project's domain is learning, but the Learning page remains available for other non-research projects.

8.6 Research

Workspace claims, status, evidence counts, source library, and contested-claim report. The focused conflict card appears when domain is research.

8.7 Artifacts

Indexed SVG/Mermaid diagrams rendered after sanitization.

8.8 Review

Approve, edit, or reject observation proposals, reflector promotion proposals, source-reverted decisions, and vault conflicts (resolved separately).

8.9 Runs

Operation usage and errors, background jobs, quarantined chunks, and Retry quarantined. Use this after fixing the observer model so failed chunks are reprocessed.

8.10 Settings

Safe live configuration (thresholds, runner model IDs, budgets, timers, cost caps, pruning). Cannot change dataDir, vaultDir, port, allowedOrigins, endpoint URLs, API-key env names, or custom headers. Also includes a redaction test, backup/restore/export, session flags, and Forget session.

The workspace selector affects workspace views. Learning and research are currently workspace-wide.

9. Desktop Memory Tab

The optional OpenCode desktop fork adds a Memory side-panel tab (enabled by default). It embeds / embed and follows the active session without reloading the iframe.

The tab provides:

• status and open-review badge;
• unobserved and observation-token gauges, daily model usage, pause/resume;
• current task, suggested next action, and the exact memory block most recently injected;
• session-scoped review items;
• learning reviews due or research conflicts for the project's domain;
• filterable observation timeline;
• search with retained-source recall;
• Remember, Flush now, and Open full dashboard.

The postMessage bridge verifies event.source, exact origin, and target origin. Insertion is capped at 16,000 characters, appends rather than replaces, switches the composer to normal mode, and never submits. Source navigation validates session/message IDs and resolves assistant messages to their parent user message. Quiz me and conflict actions insert text at the end of the composer. Action messages are honored only while the Memory tab is active and its iframe has focus.

The packaged renderer origin is oc://renderer. Development uses ELECTRON_RENDERER_URL. Both allowedOrigins and the Electron CSP must permit 127.0.0.1/localhost HTTP frames.

10. Forget, Backup, Restore, and Export

10.1 Observation forget (dashboard)

Confirmed user-global action. Selected observations and derived/similar suppressed copies are replaced with [forgotten]. Predecessor/condensed relationships, domain records, vectors, snapshots, topics, journey entries, vault output, and FTS are scrubbed. Hash/tombstone suppressions remain. Exact copies and close matches can be removed in other projects/workspaces. Optionally also delete cited conversation parts. Backups are scrubbed asynchronously.

There is no supported unforget. Restore re-applies live and backup suppressions and therefore does not resurrect forgotten content.

10.2 Session forget

Settings !’ Forget session removes observations sourced only from the session, retained source parts, copied fork descendants, artifacts, snapshots, and state. It records a permanent session suppression and marks the session excluded. The session cannot be included again.

10.3 Erasure limitations

Until WAL checkpoint, vault rewrite, and backup scrub succeed, old bytes may remain. External exports, filesystem snapshots, copied vaults, and model-provider logs are outside this mechanism. Previously exported JSON is not scrubbed after a later forget.

10.4 Backup and restore

Create backup makes a marked SQLite snapshot via VACUUM INTO in dataDir/backups. Daily backups are on by default; keep defaults to 14, and at least one backup is always retained.

Restore accepts only a file currently listed by the service, requires confirmation, and refuses missing/empty files, non-memory databases, unsupported/newer schemas, unmarked files, and snapshots with no sessions/observations/topics. Restore is logical and leaves the live database untouched on refusal. Suppressions are merged and re-applied; FTS is rebuilt.

10.5 Export JSON

Downloads observations whose status is neither forgotten nor rejected (including superseded or pending rows), plus topics, journey, concepts, claims, sources, and artifact metadata. It is a data export, not a restorable backup.

11. HTTP API Surface

Unauthenticated: GET /health (data-free compatibility probe).

Authenticated /api routes include: status, projects, workspaces, sessions, observations, search, recall, timeline, context, review (GET/POST), remember, forget and forget/session and forget/candidates, flush, reflect, pause, session flags, chunk retry and retry-quarantined, operations, jobs, commands, topics (list, get, approve, rollback), learning, research, artifacts, backups (list, create, restore), export, config (GET/POST), redact-test.

Security controls: bind 127.0.0.1 only; Host must match loopback; Origin must be the server origin or an exact allowedOrigins entry; CORS preflight rejected; POST must be JSON object "d1 MB; destructive routes require confirm:true; token comparison is timing-safe. The token is 32 random bytes stored at dataDir/token. It is not protection from other processes running as the same OS user.

12. Configuration Highlights

Global config: ~/.config/opencode-memory/config.json (XDG-style on every OS, including Windows).

Project config: .opencode/memory.json with workspace, domain (general|learning|research), name, exclude.

Runner kinds:

• opencode-session — hidden OpenCode child sessions (recommended).

• direct — OpenAI-compatible /chat/completions with JSON schema. API keys live in an environment variable named by apiKeyEnv, never in JSON. HTTPS required except loopback.

Embeddings: provider none (default) or openai-compatible /embeddings. Disabled by default. Query embedding is bounded by half of inject.deadlineMs.

Important defaults: port 4777; activationTokens 30000; bufferTokens 6000; dailyCapUsd 1; dailyTokenCap 2,000,000; inject.deadlineMs 250; captureReasoning false; observeSubagents false; pruning disabled; autoDaily backups true.

Pin runner.model to a working structured-output model (for this installation, openai/gpt-5.6-sol). The observer does not follow the chat model selector.

13. Installation and Operations

Requirements: Bun for builds/tests; Node.js 24 for the installer and node:sqlite; OpenCode v1.18.32 for desktop-fork integration.

Install: bun install, bun run build, node scripts/install-local.mjs. The installer adds an absolute file: URL of packages/plugin/dist/index.js to OpenCode's plugin array, preserving JSONC comments and backing up the previous config.

First run: restart OpenCode, open a normal session, run /memory-status, then open http://127.0.0.1:4777/app.

Data lives under ~/.local/share/opencode-memory (memory.db, token, vault, backups). Uninstall removes only the plugin entry, not data.

Verify: bun run build, typecheck, bun test across core/plugin/evals/dashboard, plus Node compatibility checks. Hermetic evals live in packages/evals/test. Real-provider evals are opt-in and cost-capped.

14. Security Summary and Known Limits

Defense in depth, not a sandbox. A process as the same OS user can read the token or fetch /app. Pattern-based redaction can miss secrets. Configured model and embedding endpoints receive redacted text. Backup erasure can remain pending after forget. Do not expose port 4777 through a proxy, container bridge, port-forward, or remote desktop URL.

SQLite, token, vault, and backups are not encrypted by this project. POSIX permissions are best-effort; Windows relies on ACLs. Vault/artifact path checks reduce accidental or model-driven path escape.

Observer confidence is model metadata, not truth. Inspect sources. Use review, edit, supersede, or forget.

15. Feature Checklist

Implemented:

- Session capture with redaction, reasoning/tool-output limits, fork lineage, and revert handling.
- Durable chunking with activation, idle, age, recovery, flush, leases, retries, and quarantine.
- Observer and reflector with structured output, validation, cost caps, and pause-on-auth.
- Observations at session, project, workspace, and user scopes with source ranges.
- Automatic prompt injection, compaction notes, optional pruning.
- Agent tools and slash commands listed in sections 5 and 6.
- Human review for shared/high-impact/mastery proposals and vault conflicts.
- Learning concepts, quiz/retention attempts, FSRS scheduling, mastery model.
- Research claims, sources, evidence, and conflict reports.
- SVG/Mermaid artifact indexing and sanitized rendering.
- Full dashboard and embedded Memory tab.

- JSON export and scrub-aware SQLite backups.
- Local loopback HTTP with Host/Origin/token/CSP controls.

Not claimed: remote multi-user sync, encrypted-at-rest storage, guaranteed secret redaction, or truth of model-generated memory.